Practice Exercises

1 File Ownership & Permissions
•Use ls -l to check the owner and group of a file. What do the columns represent?
•Verify your user ID, primary group, and additional groups using id.
•Change the owner of testfile.txt to another user.
•Change the group owner of testfile.txt to developers.

2 Modifying File Permissions
•Grant execute permission to the owner of script.sh using the symbolic method.
•Remove write permission from the group for data.txt.
•Set full permissions (rwx) for the owner and group, and read-only (r--) for others using octal notation.

3 Monitoring & Managing Processes
•Use ps aux to list running processes. What do the columns show?
•Find the process ID of a running sleep command using pgrep.
•Send a SIGTERM signal to stop the process.
•Start a long-running task in the background and bring it to the foreground.

4 System Monitoring
•Use uptime to check system load averages.
•Run free -h to check memory usage. What do the columns represent?
•Use watch -n 5 free -h to monitor memory usage every 5 seconds. How do you stop it?

Week 5 practical/lab 1 – Controlling access to files

Objective: Create users and a shared directory, then configure ownership and permissions.

Step 1: Create a group and users

sudo groupadd projectteam
sudo useradd alice
sudo useradd bob
sudo passwd alice
sudo passwd bob

Add both users to the group:

sudo usermod -aG projectteam alice
sudo usermod -aG projectteam bob

Verify:

id alice
id bob

Step 2: Create a shared directory

sudo mkdir /project
sudo chown root:projectteam /project

Verify:

ls -ld /project

Expected ownership:

root projectteam

Step 3: Give owner and group full access

sudo chmod 770 /project

Verify:

ls -ld /project

Expected:

drwxrwx---

Only root and members of `projectteam` can access it.

Step 4: Apply setgid (setuid -> 4000 , stickybit 1000)

sudo chmod g+s /project

or:

sudo chmod 2770 /project

Verify:

ls -ld /project

Expected:

drwxrws---

Files created inside should inherit `projectteam` as their group owner.

Step 5: Test group inheritance

Switch to Alice:

su - alice

Create a file:

touch /project/alice-file
ls -l /project/alice-file

Expected group owner:

projectteam

Exit:

exit

Step 6: Create a public shared directory with sticky bit

sudo mkdir /publicshare
sudo chmod 1777 /publicshare

Verify:

ls -ld /publicshare

Expected:

drwxrwxrwt

Users can create files, but they cannot normally delete files owned by other users.

Step 7: Test `umask`

umask 027
touch ~/umask-file
mkdir ~/umask-dir
ls -ld ~/umask-file ~/umask-dir

Expected:

-rw-r----- umask-file
drwxr-x--- umask-dir

Step 8: Clean up

```
sudo rm -rf /project /publicshare
sudo userdel -r alice
sudo userdel -r bob
sudo groupdel projectteam
```

Week 5 practical/lab 2 – Monitoring and managing processes

Objective: Start, monitor, suspend, resume and terminate processes.

Step 1: View current processes

```
ps
ps -ef
ps aux
```

Search for your shell:

```
ps aux | grep bash
```

Step 2: Start background processes

```
sleep 300 &
sleep 400 &
```

View them:

```
jobs
jobs -l
```

Record the job numbers and PIDs.

Step 3: Bring a job to the foreground

```
fg %1
```

Press:

Ctrl+Z

The process should become stopped.

Verify:

jobs

Step 4: Continue it in the background

bg %1

Verify:

jobs

Expected status:

Running

Step 5: Check the process state

ps -o pid,ppid,stat,cmd -C sleep

Possible state:

S

This means sleeping.

Step 6: Terminate one process normally

jobs -l
kill PID

Replace `PID` with the correct process ID.

Verify:

ps -p PID

No process should be returned.

Step 7: Force termination if required

kill -9 PID

Use this only when normal `kill` does not work.

Step 8: Start a process with lower priority

nice -n 10 sleep 500 &

Find its PID and nice value:

ps -o pid,ni,stat,cmd -C sleep

Step 9: Change the priority

renice 15 -p PID

Verify:

ps -o pid,ni,cmd -p PID

The `NI` value should now be `15`.

Step 10: End all remaining `sleep` processes

Check first:

pgrep -a sleep

Then:

pkill sleep

Verify:

pgrep -a sleep

No matching processes should remain.

Common Week 5 mistakes

Confusing file and directory permissions

For a directory:

- `r` lists names
- `w` allows creation and deletion
- `x` allows entering and accessing it

Directory write permission usually needs execute permission.

Using `chmod 777` for every problem

This gives everyone full access and is usually insecure.

Use the minimum required permissions, such as:

chmod 640 file
chmod 750 directory
chmod 770 shared-directory

Applying execute permission recursively to all files

Risky:

chmod -R a+x project/

Better:

chmod -R a+X project/

Forgetting `-R`

This changes only the specified directory:

chown student:group project/

This changes everything inside as well:

chown -R student:group project/

Confusing job numbers and PIDs

Job number:

fg %1

PID:

kill 1234

Do not use:

kill %1234

unless `1234` is actually a shell job number.

Using `Ctrl+Z` when trying to stop a process permanently

`Ctrl+Z` suspends the process.

Use:

Ctrl+C

to request termination.

Using `kill -9` immediately

Start with:

kill PID

Only use:

kill -9 PID

when normal termination fails.

Week 5 essential commands

Command	Purpose
`ls -l`	View permissions and ownership
`ls -ld`	View directory permissions
`id`	View UID, GID and groups

`chown`	Change file user owner
`chgrp`	Change group owner
`chmod`	Change permissions
`umask`	View or set default permission mask
`ps`	View processes
`ps aux`	Detailed process listing
`top`	Live process monitoring
`jobs`	View jobs in the current shell
`bg`	Continue a job in the background
`fg`	Bring a job to the foreground
`kill`	Send a signal using PID
`pkill`	Signal processes by name
`pgrep`	Find process IDs by name
`nice`	Start a process with a priority
`renice`	Change an existing process priority

setuid (Set User ID)

- Symbol: `s` in the owner's execute position.
- Numeric value: `4000`.
- On an executable file: the program runs with the **file owner's** privileges instead of the user's.
- Example: the `passwd` command uses `setuid` so regular users can update their passwords.

setgid (Set Group ID)

- Symbol: `s` in the group's execute position.
- Numeric value: `2000`.
- On an executable file: the program runs with the **file group's** privileges.

- On a directory: **new files and subdirectories inherit the directory's group**, rather than the creator's default group. This is commonly used for shared project directories.

Sticky bit

- Symbol: `t` in the others' execute position.
- Numeric value: `1000`.
- Mostly used on directories.
- Prevents users from deleting or renaming files they don't own, even if the directory is writable.
- Example: `/tmp` is typically `drwxrwxrwt`.